

7ª Edición

Comunicaciones y Redes de Computadores



PEARSON
Prentice
Hall

William Stallings

**COMUNICACIONES Y REDES
DE COMPUTADORES**
Séptima edición

Arquitectura de protocolos

2.1. ¿Por qué es necesaria una arquitectura de protocolos?

2.2. Una arquitectura de protocolos simple

Un modelo de tres capas

Arquitecturas de protocolos normalizadas

2.3. OSI

El modelo

Normalización dentro del modelo de referencia OSI

Primitivas de servicio y parámetros

Las capas de OSI

2.4. Arquitectura de protocolos TCP/IP

Las capas de TCP/IP

TCP y UDP

Funcionamiento de TCP e IP

Aplicaciones TCP/IP

Interfaces de protocolo

2.5. Lecturas recomendadas y sitios web

Sitio web recomendado

2.6. Términos clave, cuestiones de repaso y ejercicios

Términos clave

Cuestiones de repaso

Ejercicios

Apéndice 2A. El protocolo TFTP (*Trivial File Transfer Protocol*)

Introducción al TFTP

Paquetes TFTP

Ejemplo de transferencia

Errores y retardos

Sintaxis, semántica y temporización



CUESTIONES BÁSICAS

- Una arquitectura de protocolos es una estructura en capas de elementos hardware y software que facilita el intercambio de datos entre sistemas y posibilita aplicaciones distribuidas, como el comercio electrónico y la transferencia de archivos.
- En los sistemas de comunicación, en cada una de las capas de la arquitectura de protocolos se implementa uno o más protocolos comunes. Cada protocolo proporciona un conjunto de reglas para el intercambio de datos entre sistemas.
- La arquitectura de protocolos más utilizada es TCP/IP, constituida por las siguientes capas: física, acceso a la red, internet, transporte y aplicación.
- Otra arquitectura de protocolos importante es el modelo de siete capas OSI (*Open Systems Interconnection*).



En este capítulo se establece el contexto para el resto de los conceptos e ideas que se desarrollarán a lo largo del texto. Se muestra cómo los conceptos abordados en las Partes de la II a la V pertenecen a la extensa área de las comunicaciones y redes de computadores. Este capítulo se puede leer en el orden secuencial presentado o puede dejarse para el principio de las Partes III, IV o V¹.

Comenzamos presentando el concepto de arquitectura de protocolos en capas, proponiendo un ejemplo sencillo. A continuación, se define el modelo de referencia para la interconexión de sistemas abiertos OSI (*Open Systems Interconnection*). OSI es una arquitectura normalizada que frecuentemente se utiliza para describir las funciones de un sistema de comunicación, aunque en la actualidad esté implementada escasamente. Posteriormente se estudia la arquitectura de protocolos más importante, la familia de protocolos TCP/IP. TCP/IP es un concepto vinculado a Internet y es el marco de trabajo para el desarrollo de un conjunto completo de normas para las comunicaciones entre computadores. En la actualidad, todos los fabricantes de computadores dan soporte a esta arquitectura.

2.1. ¿POR QUÉ ES NECESARIA UNA ARQUITECTURA DE PROTOCOLOS?

En el intercambio de datos entre computadores, terminales y/u otros dispositivos de procesamiento, los procedimientos involucrados pueden llegar a ser bastante complejos. Considérese, por ejemplo, la transferencia de un archivo entre dos computadores. En este caso, debe haber un camino entre los dos computadores, directo o a través de una red de comunicación, pero además, normalmente se requiere la realización de las siguientes tareas adicionales:

1. El sistema fuente de información debe activar un camino directo de datos o bien debe proporcionar a la red de comunicación la identificación del sistema destino deseado.
2. El sistema fuente debe asegurarse de que el destino está preparado para recibir datos.

¹ Puede ser útil para el lector saltarse este capítulo en una primera lectura para, posteriormente, releerlo con más detenimiento antes de afrontar la lectura de la Parte V.

3. La aplicación de transferencia de archivos en el origen debe asegurarse de que el programa gestor en el destino está preparado para aceptar y almacenar el archivo para el usuario determinado.
4. Si los formatos de los dos archivos son incompatibles en ambos sistemas, uno de los dos deberá realizar una operación de traducción.

Es evidente que debe haber un alto grado de cooperación entre los computadores involucrados. En lugar de implementar toda la lógica para llevar a cabo la comunicación en un único módulo, el problema se divide en subtarefas, cada una de las cuales se realiza por separado. En una arquitectura de protocolos, los distintos módulos se disponen formando una pila vertical. Cada capa de la pila realiza el subconjunto de tareas relacionadas entre sí que son necesarias para comunicar con el otro sistema. Por lo general, las funciones más básicas se dejan a la capa inmediatamente inferior, olvidándose en la capa actual de los detalles de estas funciones. Además, cada capa proporciona un conjunto de servicios a la capa inmediatamente superior. Idealmente, las capas deberían estar definidas de forma tal que los cambios en una capa no deberían necesitar cambios en las otras.

Evidentemente, para que haya comunicación se necesitan dos entidades, por lo que debe existir el mismo conjunto de funciones en capas en los dos sistemas. La comunicación se consigue haciendo que las capas correspondientes, o **pares**, intercambien información. Las capas pares se comunican intercambiando bloques de datos que verifican una serie de reglas o convenciones denominadas **protocolo**. Los aspectos clave que definen o caracterizan a un protocolo son:

- **La sintaxis:** establece cuestiones relacionadas con el formato de los bloques de datos.
- **La semántica:** incluye información de control para la coordinación y la gestión de errores.
- **La temporización:** considera aspectos relativos a la sintonización de velocidades y secuenciación.

En el Apéndice 2A se proporciona un ejemplo específico del protocolo normalizado en Internet para la transferencia de archivos TFTP (*Trivial File Transfer Protocol*).

2.2. UNA ARQUITECTURA DE PROTOCOLOS SIMPLE

Habiendo definido el concepto de protocolo, estamos en disposición de definir el concepto de arquitectura de protocolos. A modo de ejemplo, la Figura 2.1 muestra cómo se podría implementar una aplicación de transferencia de archivos. Para ello se usan tres módulos. Las tareas 3 y 4 de la lista anterior se podrían realizar por el módulo de transferencia de archivos. Los dos módulos de ambos sistemas intercambian archivos y órdenes. Sin embargo, en vez de exigir que el módulo de transferencia se encargue de los detalles con los que se realiza el envío de datos y órdenes, dichos módulos delegan en otros módulos que ofrecen el servicio de transmisión. Cada uno de estos se encargará de asegurar que el intercambio de órdenes y datos se realice fiablemente. Entre otras cosas, estos módulos realizarán la tarea 2, por lo que, a partir de este momento, la naturaleza del intercambio entre los sistemas será independiente de la naturaleza de la red que los interconecta. Por tanto, en vez de implementar la interfaz de red en el módulo de servicio de transmisión, tiene sentido prever un módulo adicional de acceso a la red que lleve a cabo la tarea 1, interaccionando con la red.

Resumiendo, el módulo de transferencia de archivos contiene toda la lógica y funcionalidades que son exclusivas de la aplicación, como por ejemplo la transmisión de palabras de paso clave,

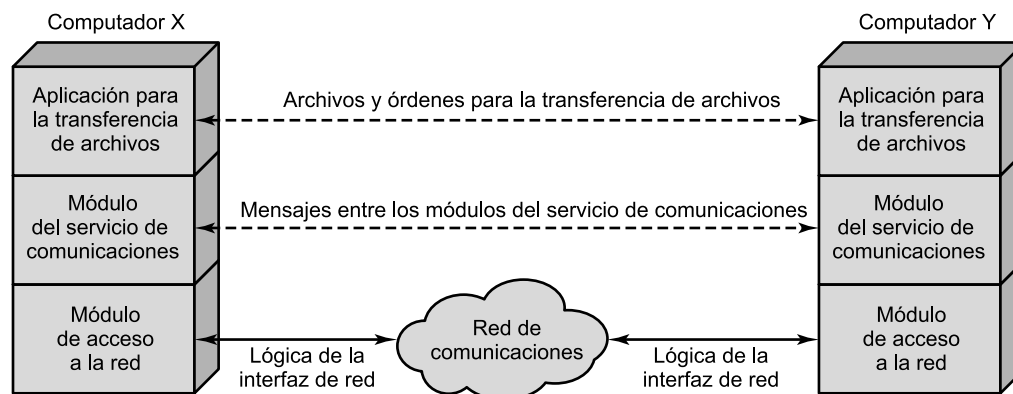


Figura 2.1. Una arquitectura simplificada para la transferencia de archivos.

de órdenes de archivo o de los registros del archivo. Es necesario que esta información (archivos y órdenes) se transmita de una forma fiable. No obstante, estos mismos requisitos de fiabilidad son compartidos por otro tipo de aplicaciones (como por ejemplo, el correo electrónico y la transferencia de documentos). Por tanto, estas funcionalidades se localizan en el módulo separado del servicio de comunicaciones de tal forma que puedan ser utilizadas por otras aplicaciones. El módulo del servicio de comunicaciones trata de asegurar que los dos computadores estén activos y preparados para la transferencia de datos, así como de seguir la pista de los datos que se intercambian, garantizando su envío. No obstante, estas tareas son independientes del tipo de red que se esté usando. Por tanto, la lógica encargada de tratar con la red se considera en un módulo separado de acceso a la misma. De esta forma, si se modifica la red que se esté usando, sólo se verá afectado el módulo de acceso a la red.

Así, en vez de disponer de un solo módulo que realice todas las tareas involucradas en la comunicación, se considera una estructura consistente en un conjunto de módulos que realizarán todas las funciones. Esta estructura se denomina **arquitectura de protocolos**. Llegados a este punto, la siguiente analogía puede ser esclarecedora. Supóngase que un ejecutivo en una oficina, digamos X, necesita enviar un documento a una oficina Y. El ejecutivo en X prepara el documento y quizá le añade una nota. Esto es análogo a las tareas que realiza la aplicación de transferencia de archivos de la Figura 2.1. A continuación, el ejecutivo le pasa el documento a un secretario o administrativo (A). El A de X mete el documento en un sobre y escribe en él la dirección postal de Y, así como el remite correspondiente a la dirección de X. Puede que en el sobre se escriba igualmente «confidencial». Lo realizado por A corresponde con el módulo del servicio de comunicaciones de la Figura 2.1. Llegados aquí, A pasa el sobre al departamento de envíos. Alguien aquí decide cómo enviar el paquete: mediante correo o mensajería. Se añaden los documentos necesarios al paquete y se realiza el envío. El departamento de envíos corresponde al módulo de acceso a la red de la Figura 2.1. Cuando el paquete llega a Y, se desencadena una serie de operaciones similares en capas. El departamento de envíos en Y recibe el paquete y lo pasa al administrativo correspondiente, dependiendo del destino que figure en el paquete. El A abre el paquete, extrae el documento y se lo pasa al ejecutivo correspondiente.

A continuación, dentro de esta sección se generalizará el ejemplo anterior para presentar una arquitectura de protocolos simplificada. Posteriormente, consideraremos ejemplos más realistas y complejos, como son TCP/IP y OSI.

UN MODELO DE TRES CAPAS

En términos muy generales, se puede afirmar que las comunicaciones involucran a tres agentes: aplicaciones, computadores y redes. Las aplicaciones se ejecutan en computadores que, generalmente, permiten múltiples aplicaciones simultáneas. Los computadores se conectan a redes y los datos a intercambiar se transfieren por la red de un computador a otro. Por tanto, la transferencia de datos desde una aplicación a otra implica, en primer lugar, la obtención de los mismos y, posteriormente, hacerlos llegar a la aplicación destino en el computador remoto.

Teniendo esto presente, parece natural estructurar las tareas de las comunicaciones en tres capas relativamente independientes: la capa de acceso a la red, la capa de transporte y la capa de aplicación.

La **capa de acceso a la red** está relacionada con el intercambio de datos entre el computador y la red a la que está conectado. El computador emisor debe proporcionar a la red la dirección del destino, de tal forma que la red pueda encaminar los datos al destino apropiado. El computador emisor necesitará hacer uso de algunos de los servicios proporcionados por la red, como por ejemplo la gestión de prioridades. Las características del software de esta capa dependerán del tipo de red que se use. Así, se han desarrollado diferentes estándares para conmutación de circuitos, conmutación de paquetes, redes de área local y otros. De esta manera, se pretende separar las funciones que tienen que ver con el acceso a la red en una capa independiente. Haciendo esto, el resto del software de comunicaciones que esté por encima de la capa de acceso a la red no tendrá que ocuparse de las características específicas de la red que se use. El mismo software de las capas superiores debería funcionar correctamente con independencia del tipo de red concreta a la que se esté conectado.

Independientemente de la naturaleza de las aplicaciones que estén intercambiando datos, es un requisito habitual que los datos se intercambien de una manera fiable. Esto es, sería deseable estar seguros de que todos los datos llegan a la aplicación destino y, además, llegan en el mismo orden en que fueron enviados. Como se verá, los mecanismos que proporcionan dicha fiabilidad son independientes de la naturaleza de las aplicaciones. Por tanto, tiene sentido concentrar todos estos procedimientos en una capa común que se comparta por todas las aplicaciones, denominada **capa de transporte**.

Finalmente, la **capa de aplicación** contiene la lógica necesaria para admitir varias aplicaciones de usuario. Para cada tipo distinto de aplicación, como por ejemplo la transferencia de archivos, se necesita un módulo independiente y con características bien diferenciadas.

Las Figuras 2.2 y 2.3 ilustran esta arquitectura sencilla. En la Figura 2.2 se muestran tres computadores conectados a una red. Cada computador contiene software en las capas de acceso a la red, de transporte y de aplicación para una o más aplicaciones. Para una comunicación con éxito, cada entidad deberá tener una dirección única. En realidad, se necesitan dos niveles de direccionamiento. Cada computador en la red debe tener una dirección de red. Esto permite a la red proporcionar los datos al computador apropiado. A su vez, cada aplicación en el computador debe tener una dirección que sea única dentro del propio computador; esto permitirá a la capa de transporte proporcionar los datos a la aplicación apropiada. Estas últimas direcciones son denominadas **puntos de acceso al servicio** (SAP, *Service Access Point*), o también **puertos**, evidenciando que cada aplicación accede individualmente a los servicios proporcionados por la capa de transporte.

La Figura 2.3 muestra cómo se comunican, mediante un protocolo, los módulos en el mismo nivel de computadores diferentes. Veamos su funcionamiento. Supóngase que una aplicación, asociada al SAP 1 en el computador X, quiere transmitir un mensaje a otra aplicación, asociada al

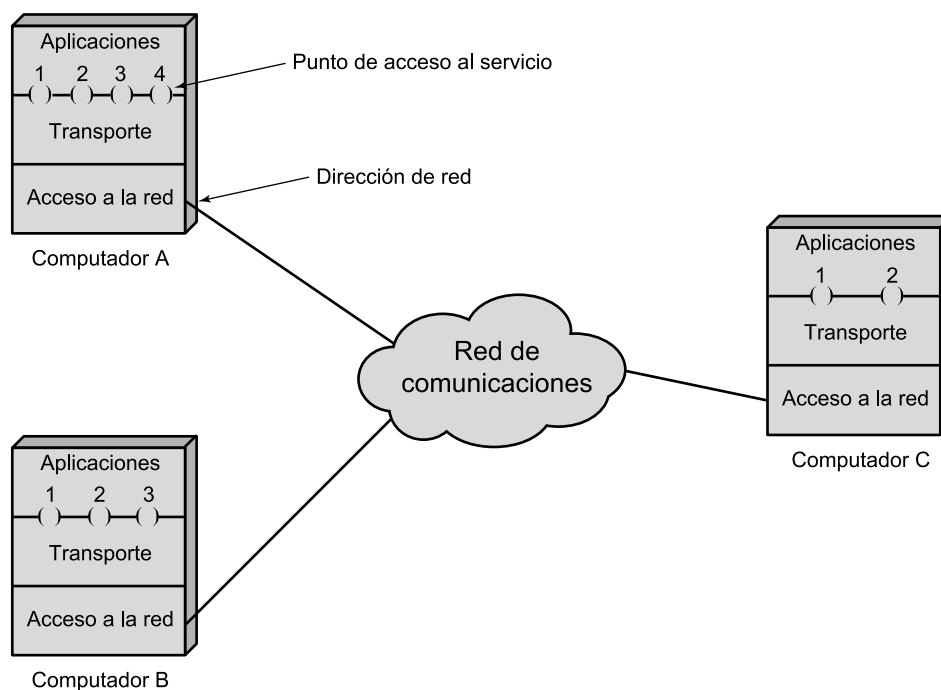


Figura 2.2. Redes y arquitecturas de protocolos.

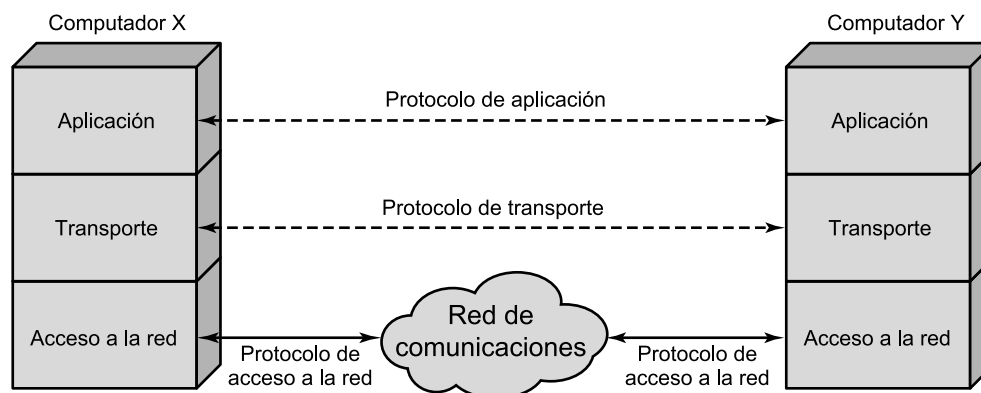


Figura 2.3. Protocolos en una arquitectura simplificada.

SAP 2 del computador Y. La aplicación en X pasa el mensaje a la capa de transporte con instrucciones para que lo envíe al SAP 2 de Y. A su vez, la capa de transporte pasa el mensaje a la capa de acceso a la red, la cual proporciona las instrucciones necesarias a la red para que envíe el mensaje a Y. Debe observarse que la red no necesita conocer la dirección del punto de acceso al servicio en el destino. Todo lo que necesita conocer es que los datos están dirigidos al computador Y.

Para controlar esta operación, se debe transmitir información de control junto a los datos del usuario, como así se muestra en la Figura 2.4. Supongamos que la aplicación emisora genera un bloque de datos y se lo pasa a la capa de transporte. Esta última puede fraccionar el bloque en unidades más pequeñas para hacerlas más manejables. A cada una de estas pequeñas unidades, la

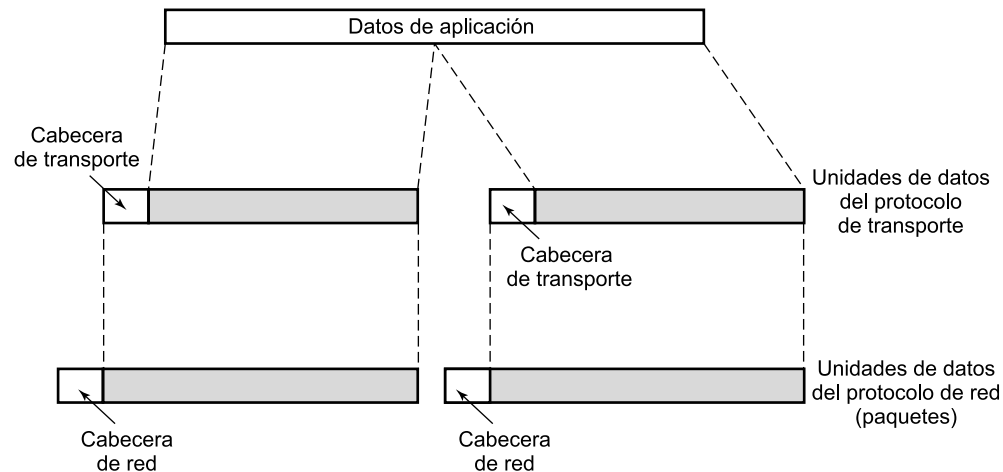


Figura 2.4. Unidades de datos de los protocolos.

capa de transporte le añadirá una cabecera, que contendrá información de control de acuerdo con el protocolo. La unión de los datos generados por la capa superior, junto con la información de control de la capa actual, se denomina **unidad de datos del protocolo** (PDU, *Protocol Data Unit*). En este caso, se denominará como **PDU de transporte**. La cabecera en cada PDU de transporte contiene información de control que será usada por el protocolo de transporte par en el computador Y. La información que se debe incluir en la cabecera puede ser por ejemplo:

- **SAP destino:** cuando la capa de transporte destino reciba la PDU de transporte, deberá saber a quién van destinados los datos.
- **Número de secuencia:** ya que el protocolo de transporte está enviando una secuencia de PDU, éstas se numerarán secuencialmente para que, si llegan desordenadas, la entidad de transporte destino sea capaz de ordenarlas.
- **Código de detección de error:** la entidad de transporte emisora debe incluir un código obtenido en función del resto de la PDU. El protocolo de transporte receptor realiza el mismo cálculo y compara los resultados con el código recibido. Si hay discrepancia se concluirá que ha habido un error en la transmisión y, en ese caso, el receptor podrá descartar la PDU y adoptar las acciones oportunas para su corrección.

El siguiente paso en la capa de transporte es pasar cada una de las PDU a la capa de red, con la instrucción de que sea transmitida al computador destino. Para satisfacer este requerimiento, el protocolo de acceso a la red debe pasar los datos a la red con una solicitud de transmisión. Como anteriormente, esta operación requiere el uso de información de control. En este caso, el protocolo de acceso a la red añade la cabecera de acceso a la red a los datos provenientes de la capa de transporte, creando así la PDU de acceso a la red. A modo de ejemplo, la cabecera debe contener la siguiente información:

- **La dirección del computador destino:** la red debe conocer a quién (qué computador de la red) debe entregar los datos.
- **Solicitud de recursos:** el protocolo de acceso a la red puede pedir a la red que realice algunas funciones, como por ejemplo, gestionar prioridades.

En la Figura 2.5 se conjugan todos estos conceptos, mostrando la interacción desarrollada entre los módulos para transferir un bloque de datos. Supongamos que el módulo de transferencia de

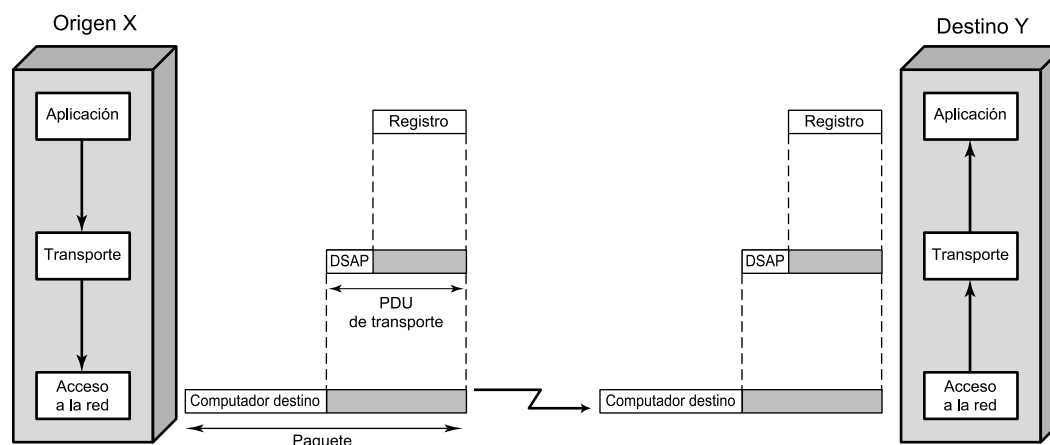


Figura 2.5. Funcionamiento de una arquitectura de protocolos.

archivos en el computador X está transfiriendo, registro a registro, un archivo al computador Y. Cada registro se pasa al módulo de la capa de transporte. Se puede describir esta acción como si se tratase de una orden o una llamada a un procedimiento. Los posibles argumentos pasados en la llamada a este procedimiento serán la dirección del destino, el SAP destino y el registro del archivo. La capa de transporte añade el punto de acceso al servicio e información de control adicional, que se agregará al registro para formar la PDU de transporte. Ésta se pasa a la capa inferior de acceso a la red mediante la llamada a otro procedimiento. En este caso, los argumentos para esta llamada serán la dirección del computador destino y la unidad de datos del protocolo de transporte. La capa de acceso a la red usará esta información para construir la PDU de red. La PDU de transporte es el campo de datos de la PDU de red, y su cabecera contendrá información relativa a las direcciones origen y destino. Nótese que la cabecera de transporte no es «visible» al nivel de acceso a la red; en otras palabras, a dicho nivel no le concierne el contenido concreto de la PDU de transporte.

La red acepta la PDU de transporte de X y la transmite a Y. El módulo de acceso a la red en Y recibe la PDU, elimina la cabecera y pasa la PDU de transporte adjunta al módulo de la capa de transporte de Y. La capa de transporte examina la cabecera de la unidad de datos del protocolo de transporte y, en función del contenido del campo de la cabecera que contenga el SAP, entregará el registro correspondiente a la aplicación pertinente, en este caso, al módulo de transferencia de archivos de Y.

ARQUITECTURAS DE PROTOCOLOS NORMALIZADAS

Cuando se desea establecer una comunicación entre computadores de diferentes fabricantes, el desarrollo del software puede convertirse en una pesadilla. Los distintos fabricantes pueden hacer uso de distintos formatos y protocolos de intercambio de datos. Incluso dentro de una misma línea de productos de un fabricante dado, los diferentes modelos pueden comunicarse de forma diferente.

Con la proliferación tanto de las comunicaciones entre computadores como de las redes, el desarrollo de software de comunicaciones de propósito específico es demasiado costoso para ser aceptable. La única alternativa para los fabricantes es adoptar e implementar un conjunto de convenciones comunes. Para que esto ocurra, es necesaria la normalización. Los estándares tienen las siguientes ventajas:

- Los fabricantes están motivados para implementar las normalizaciones con la esperanza de que, debido al uso generalizado de las normas, sus productos tendrán un mercado mayor.
- Los clientes pueden exigir que cualquier fabricante implemente los estándares.

Hay dos arquitecturas que han sido determinantes y básicas en el desarrollo de los estándares de comunicación: el conjunto de protocolos TCP/IP y el modelo de referencia de OSI. TCP/IP es, con diferencia, la arquitectura más usada. OSI, aun siendo bien conocida, nunca ha llegado a alcanzar las promesas iniciales. Además de las anteriores, hay otra arquitectura propietaria ampliamente utilizada: la SNA (*System Network Architecture*) de IBM. En lo que resta de este capítulo se estudiará OSI y TCP/IP.

2.3. OSI

Los estándares son necesarios para promover la interoperatividad entre los equipos de distintos fabricantes, así como para facilitar economías de gran escala. Debido a la complejidad que implican las comunicaciones, un solo estándar no es suficiente. En su lugar, las distintas funcionalidades deberían dividirse en partes más manejables, estructurándose en una arquitectura de comunicaciones. La arquitectura constituirá, por tanto, el marco de trabajo para el proceso de normalización. Esta línea argumental condujo a la Organización Internacional de Estandarización (ISO, *International Organization for Standardization*) en 1977 a establecer un subcomité para el desarrollo de tal arquitectura. El resultado fue el modelo de referencia OSI. Aunque los elementos esenciales del modelo se definieron rápidamente, la norma ISO final, ISO 7498, no fue publicada hasta 1984. La CCITT (en la actualidad denominada UIT-T) definió igualmente una versión técnicamente compatible con la anterior, denominada X.200.

EL MODELO

Una técnica muy aceptada para estructurar los problemas, y así fue adoptada por ISO, es la división en capas. En esta técnica, las funciones de comunicación se distribuyen en un conjunto jerárquico de capas. Cada capa realiza un subconjunto de tareas, relacionadas entre sí, de entre las necesarias para llegar a comunicarse con otros sistemas. Por otra parte, cada capa se sustenta en la capa inmediatamente inferior, la cual realizará funciones más primitivas, ocultando los detalles a las capas superiores. Una capa proporciona servicios a la capa inmediatamente superior. Idealmente, las capas deberían estar definidas para que los cambios en una capa no implicaran cambios en las otras capas. De esta forma, el problema se descompone en varios subproblemas más abordables.

La labor de ISO consistió en definir el conjunto de capas, así como los servicios a realizar por cada una de ellas. La división debería agrupar a las funciones que fueran conceptualmente próximas en un número suficiente, tal que cada capa fuese lo suficientemente pequeña, pero sin llegar a definir demasiadas para evitar así sobrecargas en el procesamiento. Las directrices generales que se adoptaron en el diseño se resumen en la Tabla 2.1. El modelo de referencia resultante tiene siete capas, las cuales son mostradas, junto a una breve definición, en la Figura 2.6. La Tabla 2.2 proporciona la justificación para la selección de las capas argumentada por ISO.

La Figura 2.7 muestra la arquitectura OSI. Cada sistema debe contener las siete capas. La comunicación se realiza entre las dos aplicaciones de los dos computadores, etiquetadas como aplicación X e Y en la figura. Si la aplicación X quiere transmitir un mensaje a la aplicación Y, invoca

Tabla 2.1. Directrices seguidas en la definición de las capas OSI (X.200).

1. No crear demasiadas capas de tal forma que la descripción e integración de las capas implique más dificultades de las necesarias.
2. Crear una separación entre capas en todo punto en el que la descripción del servicio sea reducida y el número de interacciones a través de dicha separación sea pequeña.
3. Crear capas separadas allá donde las funciones sean manifiestamente diferentes tanto en la tarea a realizar como en la tecnología involucrada.
4. Agrupar funciones similares en la misma capa.
5. Fijar las separaciones en aquellos puntos en los que la experiencia acumulada haya demostrado su utilidad.
6. Crear capas que puedan ser rediseñadas en su totalidad y los protocolos cambiados de forma drástica para aprovechar eficazmente cualquier innovación que surja tanto en la arquitectura, el hardware o tecnologías software, sin tener que modificar los servicios ofrecidos o usados por las capas adyacentes.
7. Crear una separación allá donde sea conveniente tener la correspondiente interfaz normalizada.
8. Crear una capa donde haya necesidad de un nivel distinto de abstracción (morfológico, sintáctico o semántico) a la hora de gestionar los datos.
9. Permitir que los cambios en las funciones o protocolos se puedan realizar sin afectar a otras capas.
10. Para cada capa establecer separaciones sólo con sus capas inmediatamente superiores o inferiores.

Las siguientes premisas se propusieron igualmente para definir subcapas:
11. Crear posteriores subagrupamientos y reestructurar las funciones formando subcapas dentro de una capa en aquellos casos en los que se necesiten diferentes servicios de comunicación.
12. Crear, allá donde sea necesario, dos o más subcapas con una funcionalidad común, y mínima, para permitir operar con las capas adyacentes.
13. Permitir la no utilización de una subcapa dada.

a la capa de aplicación (capa 7). La capa 7 establece una relación paritaria con la capa 7 del computador destino, usando el protocolo de la capa 7 (protocolo de aplicación). Este protocolo necesita los servicios de la capa 6, de forma tal que las dos entidades de la capa 6 usan un protocolo común y conocido, y así sucesivamente hasta llegar a la capa física, en la que realmente se transmiten los bits a través del medio físico.

Nótese que, exceptuando la capa física, no hay comunicación directa entre las capas pares. Esto es, por encima de la capa física, cada entidad de protocolo pasa los datos hacia la capa inferior contigua, para que ésta los envíe a su entidad par. Es más, el modelo OSI no requiere que los dos sistemas estén conectados directamente, ni siquiera en la capa física. Por ejemplo, para proporcionar el enlace de comunicación se puede utilizar una red de conmutación de paquetes o de conmutación de circuitos.

La Figura 2.7 también muestra cómo se usan las unidades de datos de protocolo (PDU) en la arquitectura OSI. En primer lugar, considérese la forma más habitual de implementar un protocolo.

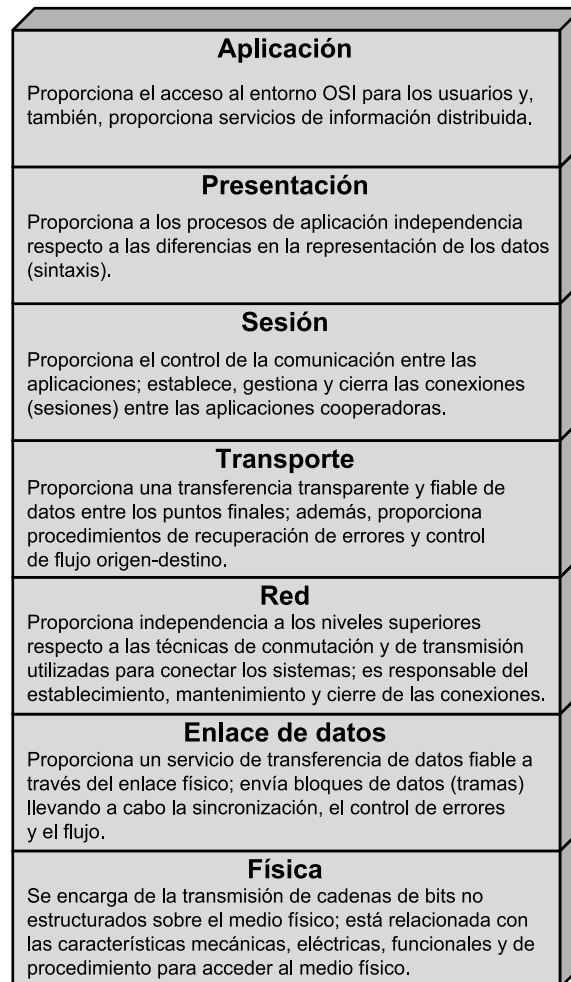


Figura 2.6. Las capas de OSI.

Cuando la aplicación X tiene un mensaje para enviar a la aplicación Y, transfiere estos datos a una entidad de la capa de aplicación. A los datos se les añade una cabecera que contiene información necesaria para el protocolo de la capa 7 (encapsulado). Seguidamente, los datos originales más la cabecera se pasan como una unidad a la capa 6. La entidad de presentación trata la unidad completa como si de datos se tratara y le añade su propia cabecera (un segundo encapsulado). Este proceso continúa hacia abajo hasta llegar a la capa 2, que normalmente añade una cabecera y una cola. La unidad de datos de la capa 2, llamada trama, se pasa al medio de transmisión mediante la capa física. En el destino, al recibir la trama, ocurre el proceso inverso. Conforme los datos ascienden, cada capa elimina la cabecera más externa, actúa sobre la información de protocolo contenida en ella y pasa el resto de la información hacia la capa inmediatamente superior.

En cada etapa del proceso, cada una de las capas puede fragmentar la unidad de datos que recibe de la capa inmediatamente superior en varias partes, de acuerdo con sus propias necesidades. Estas unidades de datos deben ser ensambladas por la capa par correspondiente antes de pasarlas a la capa superior.

Tabla 2.2. Justificación de las capas OSI (X.200)

1. Es esencial que la arquitectura permita la utilización de una variedad realista de medios físicos para la interconexión con diferentes procedimientos de control (por ejemplo, V.24, V.25, etc.). La aplicación de los principios 3, 5 y 8 (Tabla 2.1) nos conduce a la identificación de la **capa física** como la capa más baja en la arquitectura.
2. Algunos medios de comunicación físicos (por ejemplo, las líneas telefónicas) requieren técnicas específicas para usarlos, al transmitir datos entre sistemas a pesar de sufrir una tasa de error elevada (una tasa de error no aceptable para la gran mayoría de las aplicaciones). Estas técnicas específicas se utilizan en procedimientos de control del enlace de datos que han sido estudiados y normalizados durante una serie de años. También se debe reconocer que los nuevos medios de comunicación física (por ejemplo, la fibra óptica) requerirán diferentes procedimientos de control del enlace de datos. La aplicación de los principios 3, 5 y 8 nos conduce a la identificación de la **capa del enlace de datos** situada encima de la capa física en la arquitectura.
3. En la arquitectura de sistemas abiertos, algunos sistemas abiertos actuarán como el destino final de los datos. Algunos sistemas abiertos podrían actuar solamente como nodos intermedios (reenviando los datos a otros sistemas). La aplicación de los principios 3, 5 y 7 nos conduce a la identificación de la **capa de red** encima de la capa del enlace de datos. Los protocolos dependientes de la red, como por ejemplo el encaminamiento, se agrupan en esta capa. Así, la capa de red proporcionará un camino de conexión (conexión de red) entre un par de entidades de transporte incluyendo el caso en el que estén involucrados nodos intermedios.
4. El control del transporte de los datos desde el sistema final origen al sistema final destino (que no se lleva a cabo en nodos intermedios) es la última función necesaria para proporcionar la totalidad del servicio de transporte. Así, la capa superior situada justo encima de la capa de red es la **capa de transporte**. Esta capa libera a las entidades de capas superiores de cualquier preocupación sobre el transporte de datos entre ellas.
5. Existe una necesidad de organizar y sincronizar el diálogo y controlar el intercambio de datos. La aplicación de los principios 3 y 4 nos conduce a la identificación de la **capa de sesión** encima de la capa de transporte.
6. El conjunto restante de funciones de interés general son aquellas relacionadas con la representación y la manipulación de datos estructurados, para el beneficio de los programas de aplicación. La aplicación de los principios 3 y 4 nos conduce a la identificación de la **capa de presentación** encima de la capa de sesión.
7. Finalmente, existen aplicaciones consistentes en procesos de aplicación que procesan la información. Un aspecto de estos procesos de aplicación y los protocolos mediante los que se comunican comprenden la **capa de aplicación**, que es la capa más alta de la arquitectura.

NORMALIZACIÓN DENTRO DEL MODELO DE REFERENCIA OSI²

La principal motivación para el desarrollo del modelo OSI fue proporcionar un modelo de referencia para la normalización. Dentro del modelo, en cada capa se pueden desarrollar uno o más protocolos. El modelo define en términos generales las funciones que se deben realizar en cada capa y simplifica el procedimiento de la normalización ya que:

- Como las funciones de cada capa están bien definidas, para cada una de ellas, el establecimiento de normas o estándares se puede desarrollar independiente y simultáneamente. Esto acelera el proceso de normalización.

² Los conceptos que aquí se introducen son igualmente válidos para la arquitectura TCP/IP.

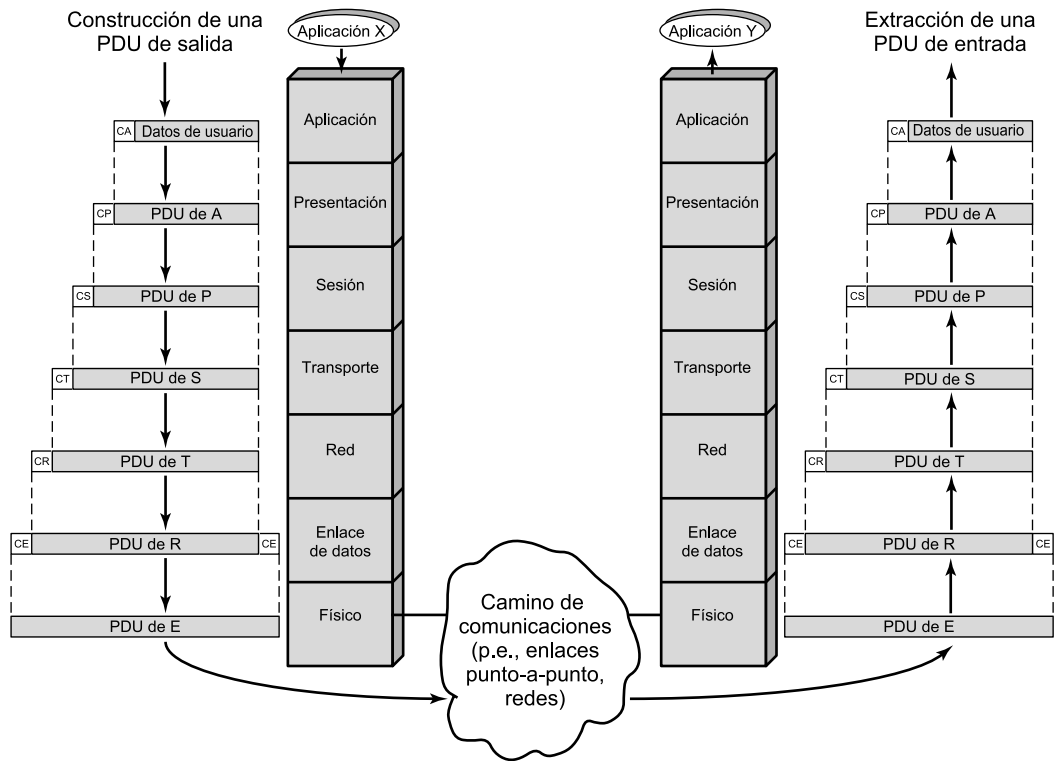


Figura 2.7. El entorno OSI.

- Como los límites entre capas están bien definidos, los cambios que se realicen en los estándares para una capa dada no afectan al software de las otras. Esto hace que sea más fácil introducir nuevas normalizaciones.

La Figura 2.8 muestra el uso del modelo de referencia OSI. La función global de comunicación se descompone en 7 capas distintas, utilizando los principios indicados en la Tabla 2.1. Estos principios esencialmente vienen a ser los mismos que rigen en el diseño modular. Esto es, la función global se descompone en una serie de módulos, haciendo que las interfaces entre módulos sean tan simples como sea posible. Además, se utiliza el principio de ocultación de la información: las capas inferiores abordan ciertos detalles de tal manera que las capas superiores sean ajenas a las particularidades de estos detalles. Dentro de cada capa, se suministra el servicio proporcionado a la capa inmediatamente superior, a la vez que se implementa el protocolo con la capa par en el sistema remoto.

La Figura 2.9 muestra de una forma más específica la naturaleza de la normalización requerida en cada capa. Existen tres elementos clave:

- **Especificación del protocolo:** dos entidades en la misma capa en sistemas diferentes cooperan e interactúan por medio del protocolo. El protocolo se debe especificar con precisión, ya que están implicados dos sistemas abiertos diferentes. Esto incluye el formato de la unidad de datos del protocolo, la semántica de todos los campos, así como la secuencia permitida de PDU.

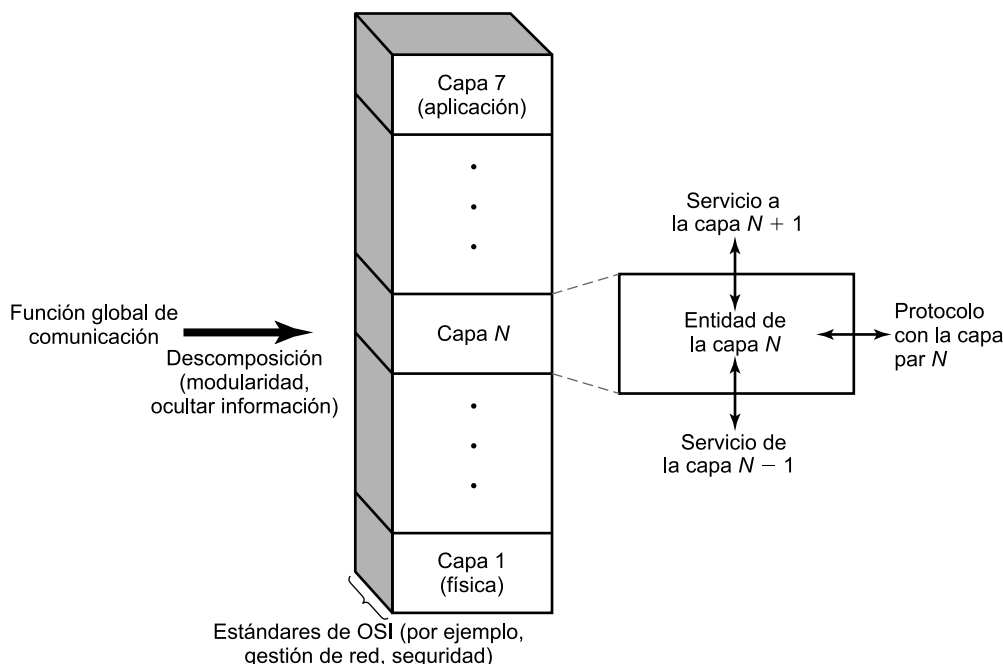


Figura 2.8. La arquitectura OSI como un modelo de referencia para las normalizaciones.

- **Definición del servicio:** además del protocolo o protocolos que operan en una capa dada, se necesitan normalizaciones para los servicios que cada capa ofrece a la capa inmediatamente superior. Normalmente, la definición de los servicios es equivalente a una descripción funcional que definiera los servicios proporcionados, pero sin especificar cómo se están proporcionando.
- **Direccionamiento:** cada capa suministra servicios a las entidades de la capa inmediatamente superior. Las entidades se identifican mediante un punto de acceso al servicio (SAP). Así, un punto de acceso al servicio de red (NSAP, *Network SAP*) identifica a una entidad de transporte usuaria del servicio de red.

En los sistemas abiertos, la necesidad de proporcionar una especificación precisa del protocolo se evidencia por sí sola. Los otros dos elementos de la lista anterior requieren más comentarios. Con respecto a la definición de servicios, la motivación para proporcionar sólo una definición funcional es la siguiente. Primero, la interacción entre capas adyacentes tiene lugar dentro de los confines de un único sistema abierto y, por tanto, le incumbe sólo a él. Así, mientras las capas pares en diferentes sistemas proporcionen los mismos servicios a las capas inmediatamente superiores, los detalles de cómo se suministran los servicios pueden diferir de un sistema a otro sin que ello implique pérdida de interoperatividad. Segundo, es frecuente que las capas adyacentes estén implementadas en el mismo procesador. En estas circunstancias, sería interesante dejar libre al programador del sistema para que utilice el hardware y el sistema operativo para que proporcionen una interfaz que sea lo más eficiente posible.

En lo que se refiere al direccionamiento, la utilización de un mecanismo de direccionamiento en cada capa, materializado en el SAP, permite que cada capa multiplexe varios usuarios de la capa inmediatamente superior. La multiplexación puede que no se lleve a cabo en todos los niveles. No obstante, el modelo lo permite.

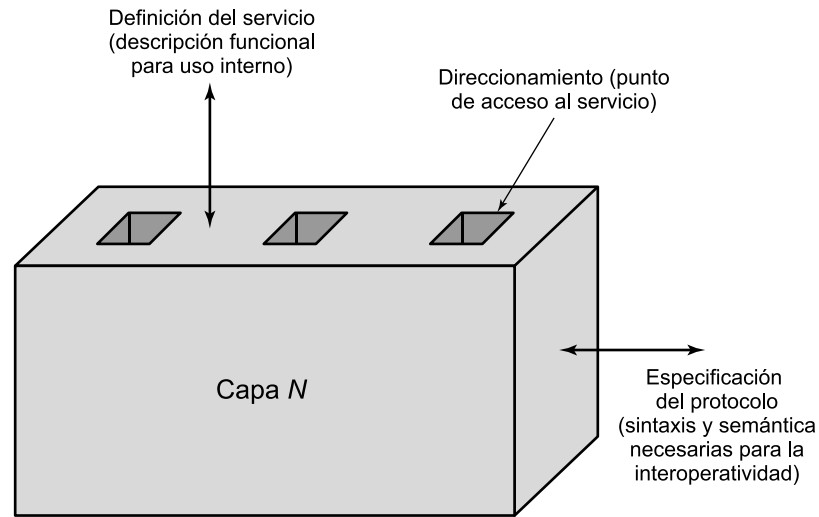


Figura 2.9. Elementos a normalizar en cada capa.

PARÁMETROS Y PRIMITIVAS DE SERVICIO

En la arquitectura OSI los servicios entre capas adyacentes se describen en términos de primitivas y mediante los parámetros involucrados. Una primitiva especifica la función que se va a llevar a cabo y los parámetros se utilizan para pasar datos e información de control. La forma concreta que adopte la primitiva dependerá de la implementación. Un ejemplo es una llamada a un procedimiento.

Para definir las interacciones entre las capas adyacentes de la arquitectura se utilizan cuatro tipos de primitivas (X.210). Éstas se definen en la Tabla 2.3. En la Figura 2.10a se muestra la ordenación temporal de estos eventos. A modo de ejemplo, considere la transferencia de datos desde una entidad (N) a su entidad par (N) en otro sistema. En esta situación se verifican los siguientes hechos:

1. La entidad origen (N) invoca a su entidad ($N - 1$) con una primitiva de *solicitud*. Asociados a esta primitiva están los parámetros necesarios, como por ejemplo, los datos que se van a transmitir y la dirección destino.

Tabla 2.3. Tipos de primitivas de servicio.

SOLICITUD	Primitiva emitida por el usuario del servicio para invocar algún servicio y pasar los parámetros necesarios para especificar completamente el servicio solicitado.
INDICACIÓN	Primitiva emitida por el proveedor del servicio para: 1. indicar que ha sido invocado un procedimiento por el usuario de servicio par en la conexión y para suministrar los parámetros asociados, o 2. notificar al usuario del servicio una acción iniciada por el suministrador.
RESPUESTA	Primitiva emitida por el usuario del servicio para confirmar o completar algún procedimiento invocado previamente mediante una indicación a ese usuario.
CONFIRMACIÓN	Primitiva emitida por el proveedor del servicio para confirmar o completar algún procedimiento invocado previamente mediante una solicitud por parte del usuario del servicio.

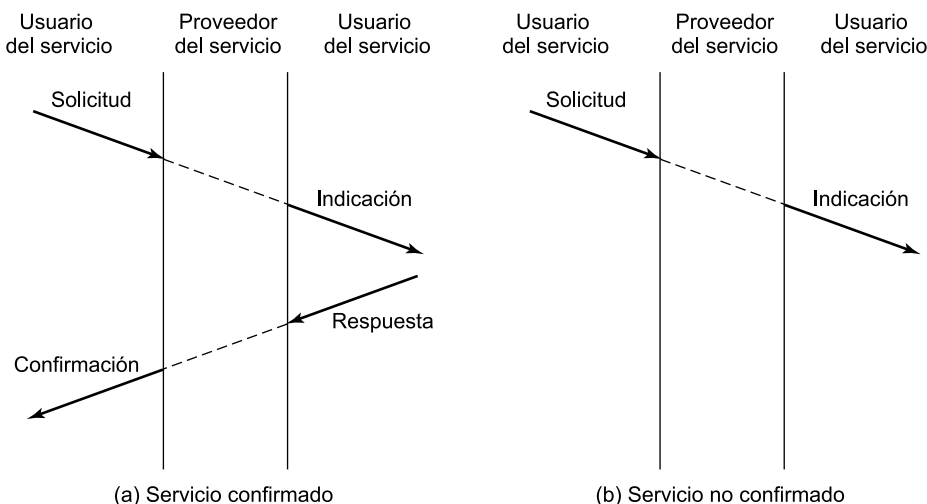


Figura 2.10. Diagramas temporales de las primitivas de servicio.

2. La entidad origen ($N - 1$) prepara una PDU ($N - 1$) para enviársela a su entidad par ($N - 1$).
3. La entidad destino ($N - 1$) entrega los datos al destino apropiado (N) a través de la primitiva de *indicación*, que incluye como parámetros los datos y la dirección origen.
4. Si se requiere una confirmación, la entidad destino (N) emite una primitiva de *respuesta* a su entidad ($N-1$).
5. La entidad ($N - 1$) convierte la confirmación en una PDU ($N - 1$).
6. La confirmación se entrega a la entidad (N) a través de una primitiva de *confirmación*.

Esta secuencia de eventos se conoce como un **servicio confirmado**, ya que el que inicia la transferencia recibe una confirmación de que el servicio solicitado ha tenido el efecto deseado en el otro extremo. Si solamente se invocan las primitivas de solicitud e indicación (correspondientes a los pasos 1 a 3), entonces se denomina **servicio no confirmado**: la entidad que inicia la transferencia no recibe confirmación de que la acción solicitada haya tenido lugar (Figura 2.10b).

LAS CAPAS DE OSI

En esta sección se estudian brevemente cada una de las capas y, donde sea pertinente, se proporcionan ejemplos de normalizaciones para los protocolos de estas capas.

Capa física

La capa física se encarga de la interfaz física entre los dispositivos. Además, define las reglas que rigen en la transmisión de los bits. La capa física tiene cuatro características importantes:

- **Mecánicas:** relacionadas con las propiedades físicas de la interfaz con el medio de transmisión. Normalmente, dentro de estas características se incluye la especificación del conector que transmite las señales a través de conductores. A estos últimos se les denominan circuitos.
- **Eléctricas:** especifican cómo se representan los bits (por ejemplo, en términos de niveles de tensión), así como su velocidad de transmisión.

- **Funcionales:** especifican las funciones que realiza cada uno de los circuitos de la interfaz física entre el sistema y el medio de transmisión.
- **De procedimiento:** especifican la secuencia de eventos que se llevan a cabo en el intercambio del flujo de bits a través del medio físico.

En el Capítulo 6 se estudian con detalle los protocolos de la capa física. Algunos ejemplos de estándares de esta capa son el EIA-232-F y algunas secciones de los estándares de comunicaciones inalámbricas y LAN.

Capa de enlace de datos

Mientras que la capa física proporciona exclusivamente un servicio de transmisión de datos, la capa de enlace de datos intenta hacer que el enlace físico sea fiable. Además proporciona los medios para activar, mantener y desactivar el enlace. El principal servicio proporcionado por la capa de enlace de datos a las capas superiores es el de detección y control de errores. Así, si se dispone de un protocolo en la capa de enlace de datos completamente operativo, la capa adyacente superior puede suponer que la transmisión está libre de errores. Sin embargo, si la comunicación se realiza entre dos sistemas que no estén directamente conectados, la conexión constará de varios enlaces de datos en serie, cada uno operando independientemente. Por tanto, en este último caso, la capa superior no estará libre de la responsabilidad del control de errores.

El Capítulo 7 se dedica a los protocolos de enlace de datos. Algunos ejemplos de estándares en esta capa son HDLC y LLC.

Capa de red

La capa de red realiza la transferencia de información entre sistemas finales a través de algún tipo de red de comunicación. Libera a las capas superiores de la necesidad de tener conocimiento sobre la transmisión de datos subyacente y las tecnologías de conmutación utilizadas para conectar los sistemas. En esta capa, el computador establecerá un diálogo con la red para especificar la dirección destino y solicitar ciertos servicios, como por ejemplo, la gestión de prioridades.

Existe un amplio abanico de posibilidades para que los servicios de comunicación intermedios sean gestionados por la capa de red. En el extremo más sencillo están los enlaces punto-a-punto directos entre estaciones. En este caso no se necesita capa de red, ya que la capa de enlace de datos puede proporcionar las funciones de gestión del enlace necesarias.

Siguiendo en orden de complejidad creciente, podemos considerar un sistema conectado a través de una única red, como una red de conmutación de circuitos o de conmutación de paquetes. Un ejemplo de esta situación es el nivel de paquetes del estándar X.25. La Figura 2.11 muestra cómo la presencia de una red se encuadra dentro de la arquitectura OSI. Las tres capas inferiores están relacionadas con la conexión y la comunicación con la red. Los paquetes creados por el sistema final pasan a través de uno o más nodos de la red, que actúan como retransmisores entre los dos sistemas finales. Los nodos de la red implementan las capas 1 a 3 de la arquitectura. En la figura anterior se muestran dos sistemas finales conectados a través de un único nodo de red. La capa 3 en el nodo realiza las funciones de conmutación y encaminamiento. Dentro del nodo, existen dos capas del enlace de datos y dos capas físicas, correspondientes a los enlaces con los dos

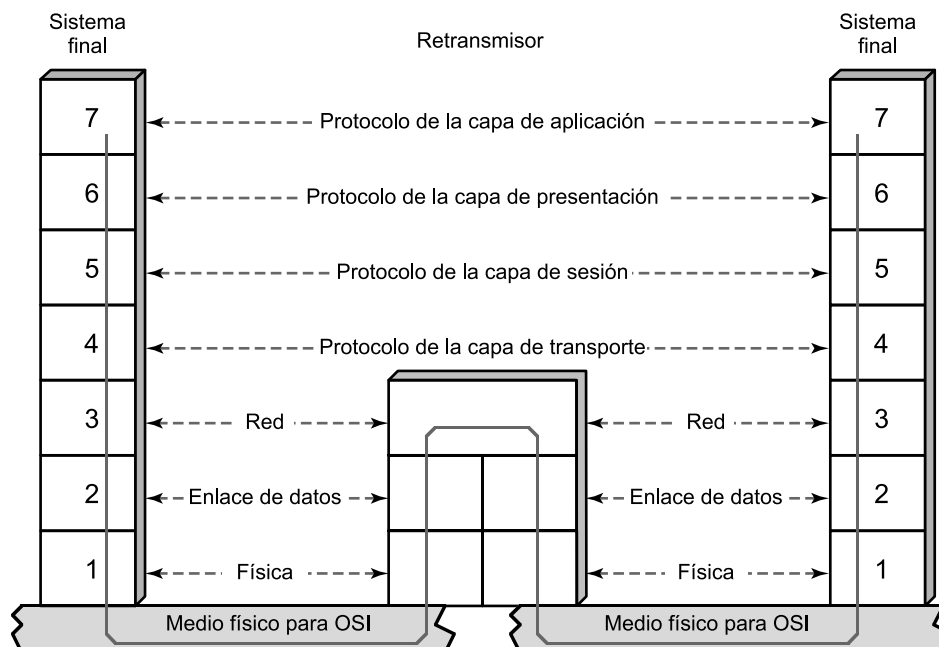


Figura 2.11. Utilización de un retransmisor.

sistemas finales. Cada capa del enlace de datos (y física) opera independientemente para proporcionar el servicio a la capa de red sobre su respectivo enlace. Las cuatro capas superiores son protocolos «extremo-a-extremo» entre los sistemas finales.

En el otro extremo de complejidad, una configuración para la capa de red puede consistir en dos sistemas finales que necesitan comunicarse sin estar conectados a la misma red. Más bien, supondremos que están conectados a redes que, directa o indirectamente, están conectadas entre sí. Este caso requiere el uso de alguna técnica de interconexión entre redes. Estas técnicas se estudiarán en el Capítulo 18.

Capa de transporte

La capa de transporte proporciona un mecanismo para intercambiar datos entre sistemas finales. El servicio de transporte orientado a conexión asegura que los datos se entregan libres de errores, en orden y sin pérdidas ni duplicaciones. La capa de transporte también puede estar involucrada en la optimización del uso de los servicios de red, y en proporcionar la calidad del servicio solicitada. Por ejemplo, la entidad de sesión puede solicitar una tasa máxima de error determinada, un retardo máximo, una prioridad y un nivel de seguridad dado.

El tamaño y la complejidad de un protocolo de transporte dependen de cómo de fiables sean los servicios de red y las redes subyacentes. Consecuentemente, ISO ha desarrollado una familia de cinco protocolos de transporte normalizados, cada uno de ellos especificado para un determinado servicio subyacente. En la arquitectura de protocolos TCP/IP se han especificado dos protocolos para la capa de transporte: el orientado a conexión, TCP (Protocolo de Control de la Transmisión, *Transmission Control Protocol*) y el no orientado a conexión UDP (Protocolo de Datagrama de Usuario, *User Datagram Protocol*).

Capa de sesión

Las cuatro capas inferiores del modelo OSI proporcionan un medio para el intercambio fiable de datos permitiendo, a su vez, distintos niveles de calidad de servicio. Para muchas aplicaciones, este servicio básico es, a todas luces, insuficiente. Por ejemplo, una aplicación de acceso a un terminal remoto puede requerir un diálogo *half-duplex*. Por el contrario, una aplicación para el procesamiento de transacciones puede necesitar la inclusión de puntos de comprobación en el flujo de transferencia para poder hacer operaciones de respaldo y recuperación. De igual manera, otra aplicación para procesar mensajes puede requerir la posibilidad de interrumpir el diálogo, generar nuevos mensajes y, posteriormente, continuar el diálogo desde donde se interrumpió.

Todas estas capacidades se podrían incorporar en las aplicaciones de la capa 7. Sin embargo, ya que todas estas herramientas para el control del diálogo son ampliamente aplicables, parece lógico organizarlas en una capa separada, denominada capa de sesión.

La capa de sesión proporciona los mecanismos para controlar el diálogo entre las aplicaciones de los sistemas finales. En muchos casos, los servicios de la capa de sesión son parcialmente, o incluso, totalmente prescindibles. No obstante, en algunas aplicaciones su utilización es ineludible. La capa de sesión proporciona los siguientes servicios:

- **Control del diálogo:** éste puede ser simultáneo en los dos sentidos (*full-duplex*) o alternado en ambos sentidos (*half-duplex*).
- **Agrupamiento:** el flujo de datos se puede marcar para definir grupos de datos. Por ejemplo, si una empresa o almacén está transmitiendo los datos correspondientes a las ventas hacia una oficina regional, éstos se pueden marcar de tal manera que se indique por grupos el final de las ventas realizadas en cada departamento. Este servicio permitiría que el computador destino calcule los totales de las ventas realizadas en cada departamento.
- **Recuperación:** la capa de sesión puede proporcionar un procedimiento de puntos de comprobación, de forma que si ocurre algún tipo de fallo entre puntos de comprobación, la entidad de sesión puede retransmitir todos los datos desde el último punto de comprobación.

ISO ha definido una normalización para la capa de sesión que incluye como opciones los servicios que se acaban de describir.

Capa de presentación

La capa de presentación define el formato de los datos que se van a intercambiar entre las aplicaciones y ofrece a los programas de aplicación un conjunto de servicios de transformación de datos. La capa de presentación define la sintaxis utilizada entre las entidades de aplicación y proporciona los medios para seleccionar y modificar la representación utilizada. Algunos ejemplos de servicios específicos que se pueden realizar en esta capa son los de compresión y cifrado de datos.

Capa de aplicación

La capa de aplicación proporciona a los programas de aplicación un medio para que accedan al entorno OSI. A esta capa pertenecen las funciones de administración y los mecanismos genéricos necesarios para la implementación de aplicaciones distribuidas. Además, en esta capa también residen las aplicaciones de uso general como, por ejemplo, la transferencia de archivos, el correo electrónico y el acceso desde terminales a computadores remotos, entre otras.

2.4. LA ARQUITECTURA DE PROTOCOLOS TCP/IP

La arquitectura de protocolos TCP/IP es resultado de la investigación y desarrollo llevados a cabo en la red experimental de conmutación de paquetes ARPANET, financiada por la Agencia de Proyectos de Investigación Avanzada para la Defensa (DARPA, *Defense Advanced Research Projects Agency*), y se denomina globalmente como la familia de protocolos TCP/IP. Esta familia consiste en una extensa colección de protocolos que se han especificado como estándares de Internet por parte de IAB (*Internet Architecture Board*).

LAS CAPAS DE TCP/IP

El modelo TCP/IP estructura el problema de la comunicación en cinco capas relativamente independientes entre sí:

- Capa física.
- Capa de acceso a la red.
- Capa internet.
- Capa extremo-a-extremo o de transporte.
- Capa de aplicación.

La **capa física** define la interfaz física entre el dispositivo de transmisión de datos (por ejemplo, la estación de trabajo o el computador) y el medio de transmisión o red. Esta capa se encarga de la especificación de las características del medio de transmisión, la naturaleza de las señales, la velocidad de datos y cuestiones afines.

La **capa de acceso a la red** es responsable del intercambio de datos entre el sistema final (servidor, estación de trabajo, etc.) y la red a la cual está conectado. El emisor debe proporcionar a la red la dirección del destino, de tal manera que ésta pueda encaminar los datos hasta el destino apropiado. El emisor puede requerir ciertos servicios que pueden ser proporcionados por el nivel de red, por ejemplo, solicitar una determinada prioridad. El software en particular que se use en esta capa dependerá del tipo de red que se disponga. Así, se han desarrollado, entre otros, diversos estándares para la conmutación de circuitos, la conmutación de paquetes (por ejemplo, retransmisión de tramas) y para las redes de área local (por ejemplo, Ethernet). Por tanto, tiene sentido separar en una capa diferente todas aquellas funciones que tengan que ver con el acceso a la red. Haciendo esto, el software de comunicaciones situado por encima de la capa de acceso a la red no tendrá que ocuparse de los detalles específicos de la red a utilizar. El software de las capas superiores debería, por tanto, funcionar correctamente con independencia de la red a la que el computador esté conectado.

Para sistema finales conectados a la misma red, la capa de acceso a la red está relacionada con el acceso y encaminamiento de los datos. En situaciones en las que los dos dispositivos estén conectados a redes diferentes, se necesitarán una serie de procedimientos que permitan que los datos atraviesen las distintas redes interconectadas. Ésta es la función de la **capa internet**. El protocolo internet (IP, *Internet Protocol*) se utiliza en esta capa para ofrecer el servicio de encaminamiento a través de varias redes. Este protocolo se implementa tanto en los sistemas finales como en los encaminadores intermedios. Un encaminador es un procesador que conecta dos redes y cuya función principal es retransmitir datos desde una red a otra siguiendo la ruta adecuada para alcanzar al destino.

Independientemente de la naturaleza de las aplicaciones que estén intercambiando datos, es usual requerir que los datos se intercambien de forma fiable. Esto es, sería deseable asegurar que todos los datos llegaran a la aplicación destino y en el mismo orden en el que fueron enviados. Como se estudiará más adelante, los mecanismos que proporcionan esta fiabilidad son esencialmente independientes de la naturaleza intrínseca de las aplicaciones. Por tanto, tiene sentido agrupar todos estos mecanismos en una capa común compartida por todas las aplicaciones; ésta se denomina **capa extremo-a-extremo**, o **capa de transporte**. El protocolo para el control de la transmisión, TCP (*Transmission Control Protocol*), es el más utilizado para proporcionar esta funcionalidad.

Finalmente, la **capa de aplicación** contiene toda la lógica necesaria para posibilitar las distintas aplicaciones de usuario. Para cada tipo particular de aplicación, como por ejemplo, la transferencia de archivos, se necesitará un módulo bien diferenciado.

La Figura 2.12 muestra las capas de las arquitecturas OSI y TCP/IP, indicando la posible correspondencia en términos de funcionalidad entre ambas.

TCP Y UDP

La mayor parte de aplicaciones que se ejecutan usando la arquitectura TCP/IP usan como protocolo de transporte TCP. TCP proporciona una conexión fiable para transferir los datos entre las aplicaciones. Una conexión es simplemente una asociación lógica de carácter temporal entre dos entidades de sistemas distintos. Cada PDU de TCP, denominada **segmento TCP**, contiene en la cabecera la identificación de los puertos origen y destino, los cuales corresponden con los puntos de acceso al servicio (SAP) de la arquitectura OSI. Los valores de los puertos identifican a los respectivos usuarios (aplicaciones) de las dos entidades TCP. Una conexión lógica alude a un par de puertos. Durante la conexión, cada entidad seguirá la pista de los segmentos TCP que vengan y vayan hacia la otra entidad, para así regular el flujo de segmentos y recuperar aquellos que se pierdan o dañen.

Además del protocolo TCP, la arquitectura TCP/IP usa otro protocolo de transporte: el protocolo de datagramas de usuario, UDP (*User Datagram Protocol*). UDP no garantiza la entrega, la

OSI	TCP/IP
Aplicación	Aplicación
Presentación	
Sesión	
Transporte	Transporte (origen-destino)
Red	Internet
Enlace de datos	Acceso a la red
Física	Física

Figura 2.12. Comparación entre las arquitecturas de protocolos TCP/IP y OSI.

conservación del orden secuencial, ni la protección frente duplicados. UDP posibilita el envío de mensajes entre aplicaciones con la complejidad mínima. Algunas aplicaciones orientadas a transacciones usan UDP. Un ejemplo es SNMP (*Simple Network Management Protocol*), el protocolo normalizado para la gestión en las redes TCP/IP. Debido a su carácter no orientado a conexión, UDP en realidad tiene poca tarea que hacer. Básicamente, su cometido es añadir a IP la capacidad de identificar los puertos.

FUNCIONAMIENTO DE TCP E IP

La Figura 2.13 muestra cómo se configuran los protocolos TCP/IP. Para poner de manifiesto que el conjunto total de recursos para la comunicación puede estar formado por varias redes, a dichas redes constituyentes se les denomina **subredes**. Para conectar un computador a una subred se utiliza algún tipo de protocolo de acceso, por ejemplo, Ethernet. Este protocolo permite al computador enviar datos a través de la subred a otro computador o, en caso de que el destino final esté en otra subred, a un dispositivo de encaminamiento que los retransmitirá. IP se implementa en todos los sistemas finales y dispositivos de encaminamiento. Actúa como un porteador que transportara bloques de datos desde un computador hasta otro, a través de uno o varios dispositivos de encaminamiento. TCP se implementa solamente en los sistemas finales, donde supervisa los bloques de datos para asegurar que todos se entregan de forma fiable a la aplicación apropiada.

Para tener éxito en la transmisión, cada entidad en el sistema global debe tener una única dirección. En realidad, se necesitan dos niveles de direccionamiento. Cada computador en una subred dada debe tener una dirección de internet única que permita enviar los datos al computador adecuado. Además, cada proceso que se ejecute dentro de un computador dado debe tener, a su vez,

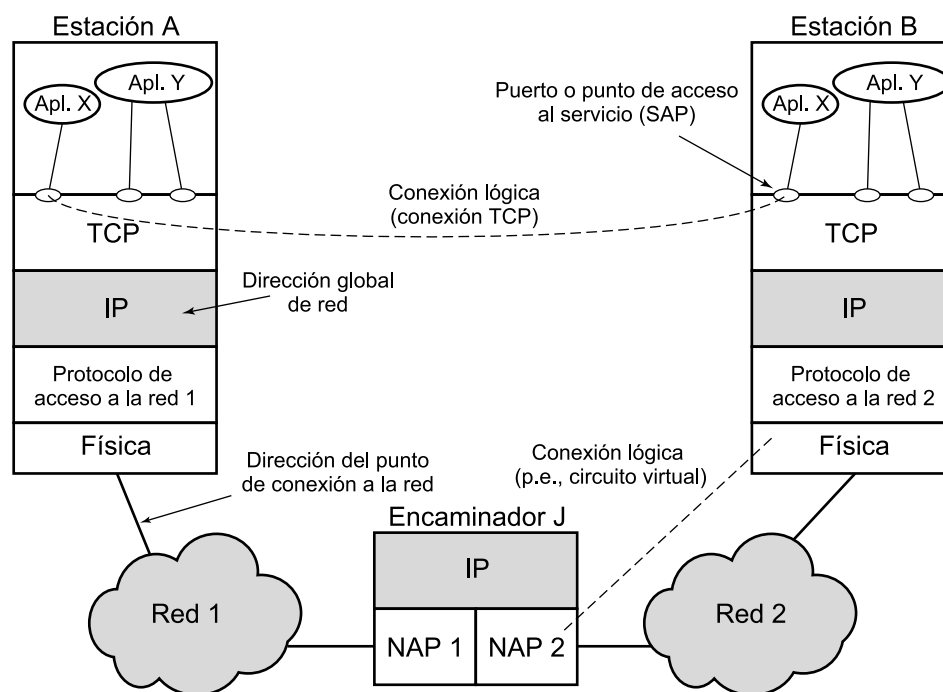


Figura 2.13. Conceptos de TCP/IP.

una dirección que sea única dentro del mismo. Esto permite al protocolo extremo-a-extremo (TCP) entregar los datos al proceso adecuado. Estas últimas direcciones se denominan puertos.

A continuación se va a describir paso a paso un sencillo ejemplo. Supóngase que un proceso, asociado al puerto 1 en el computador A, desea enviar un mensaje a otro proceso, asociado al puerto 2 del computador B. El proceso en A pasa el mensaje a TCP con la instrucción de enviarlo al puerto 2 del computador B. TCP pasa el mensaje a IP con la instrucción de enviarlo al computador B. Obsérvese que no es necesario comunicarle a IP la identidad del puerto destino. Todo lo que necesita saber es que los datos van dirigidos al computador B. A continuación, IP pasa el mensaje a la capa de acceso a la red (por ejemplo, a la lógica de Ethernet) con el mandato expreso de enviarlo al dispositivo de encaminamiento J (el primer salto en el camino hacia B).

Para controlar esta operación se debe transmitir información de control junto con los datos de usuario, como así se sugiere en la Figura 2.14. Supongamos que el proceso emisor genera un bloque de datos y lo pasa a TCP. TCP puede que divida este bloque en fragmentos más pequeños para hacerlos más manejables. A cada uno de estos fragmentos le añade información de control, denominada cabecera TCP, formando un **segmento TCP**. La información de control la utilizará la entidad par TCP en el computador B. Entre otros, en la cabecera se incluyen los siguientes campos:

- **Puerto destino:** cuando la entidad TCP en B recibe el segmento, debe conocer a quién se le deben entregar los datos.
- **Número de secuencia:** TCP numera secuencialmente los segmentos que envía a un puerto destino dado para que, si llegan desordenados, la entidad TCP en B pueda reordenarlos.
- **Suma de comprobación:** la entidad emisora TCP incluye un código calculado en función del resto del segmento. La entidad receptora TCP realiza el mismo cálculo y compara el resultado con el código recibido. Si se observa alguna discrepancia implicará que ha habido algún error en la transmisión.

A continuación, TCP pasa cada segmento a IP con instrucciones para que los transmita a B. Estos segmentos se transmitirán a través de una o varias subredes y serán retransmitidos en uno

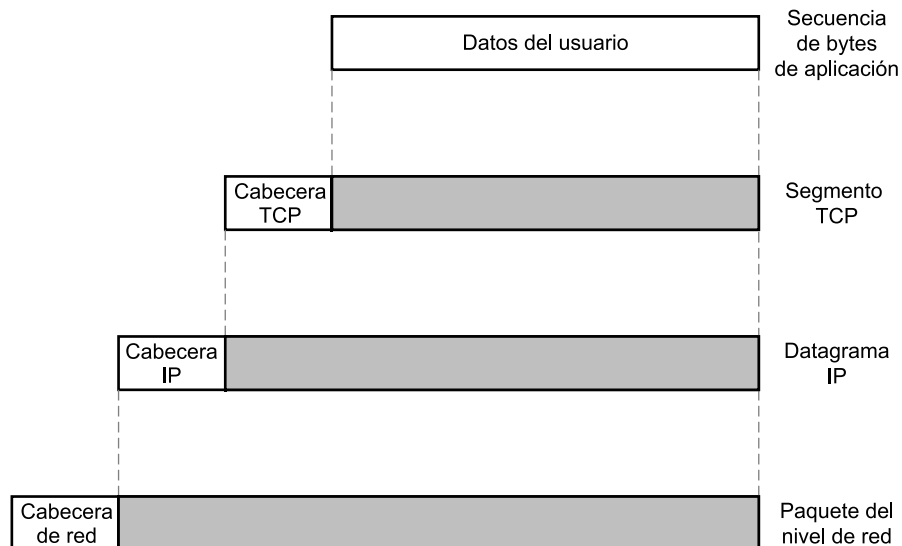


Figura 2.14. Unidades de datos de protocolo en la arquitectura TCP/IP.

o más dispositivos de encaminamiento intermedios. Esta operación también requiere el uso de información de control. Así, IP añade una cabecera de información de control a cada segmento para formar lo que se denomina un **datagrama IP**. En la cabecera IP, además de otros campos, se incluirá la dirección del computador destino (en nuestro ejemplo B).

Finalmente, cada datagrama IP se pasa a la capa de acceso a la red para que se envíe a través de la primera subred. La capa de acceso a la red añade su propia cabecera, creando un paquete, o trama. El paquete se transmite a través de la subred al dispositivo de encaminamiento J. La cabecera del paquete contiene la información que la subred necesita para transferir los datos. La cabecera puede contener, entre otros, los siguientes campos:

- **Dirección de la subred destino:** la subred debe conocer a qué dispositivo se debe entregar el paquete.
- **Funciones solicitadas:** el protocolo de acceso a la red puede solicitar la utilización de ciertas funciones ofrecidas por la subred, por ejemplo, la utilización de prioridades.

En el dispositivo de encaminamiento J, la cabecera del paquete se elimina y, posteriormente, se examina la cabecera IP. El módulo IP del dispositivo de encaminamiento direcciona el paquete a través de la subred 2 hacia B basándose en la dirección destino que contenga la cabecera IP. Para hacer esto, se le añade al datagrama una cabecera de acceso a la red.

Cuando se reciben los datos en B, ocurre el proceso inverso. En cada capa se elimina la cabecera correspondiente y el resto se pasa a la capa inmediatamente superior, hasta que los datos de usuario originales alcancen al proceso destino.

Como nota final, recuérdese que el nombre genérico del bloque de datos intercambiado en cualquier nivel se denomina **unidad de datos del protocolo** (PDU, *Protocol Data Unit*). Consecuentemente, el segmento TCP es la PDU del protocolo TCP.

APLICACIONES TCP/IP

Se han normalizado una serie de aplicaciones para funcionar por encima de TCP. A continuación se mencionan tres de las más importantes.

El **protocolo simple de transferencia de correo (SMTP, Simple Mail Transfer Protocol)** proporciona una función básica de correo electrónico. Este protocolo establece un mecanismo para transferir mensajes entre computadores remotos. Entre las características de SMTP cabe destacar la utilización de listas de mensajería, la gestión de acuses de recibo y el reenvío de mensajes. El protocolo SMTP no especifica cómo se crean los mensajes. Para este fin se necesita un programa de correo electrónico nativo o un editor local. Una vez que se ha creado el mensaje, SMTP lo acepta y, utilizando TCP, lo envía al módulo SMTP del computador remoto. En el receptor, el módulo SMTP utilizará su aplicación de correo electrónico local para almacenar el mensaje recibido en el buzón de correo del usuario destino.

El **protocolo de transferencia de archivos (FTP, File Transfer Protocol)** se utiliza para enviar archivos de un sistema a otro bajo el control del usuario. Se permite transmitir archivos tanto de texto como en binario. Además, el protocolo permite controlar el acceso de los usuarios. Cuando un usuario solicita la transferencia de un archivo, FTP establece una conexión TCP con el sistema destino para intercambiar mensajes de control. Esta conexión permite al usuario transmitir su identificador y contraseña, además de la identificación del archivo junto con las acciones a realizar sobre el mismo. Una vez que el archivo se haya especificado y su transferencia haya sido aceptada, se establecerá una segunda conexión TCP a través de la cual se materializará la transferencia. El

archivo se transmite a través de la segunda conexión, sin necesidad de enviar información extra o cabeceras generadas por la capa de aplicación. Cuando la transferencia finaliza, se utiliza la conexión de control para indicar la finalización. Además, esta misma conexión estará disponible para aceptar nuevas órdenes de transferencia.

TELNET facilita la realización de conexiones remotas, mediante las cuales el usuario en un terminal o computador personal se conecta a un computador remoto y trabaja como si estuviera conectado directamente a ese computador. El protocolo se diseñó para trabajar con terminales poco sofisticados en modo *scroll* (avance de pantalla). En realidad, TELNET se implementa en dos módulos: el usuario TELNET interactúa con el módulo de E/S para comunicarse con un terminal local. Este convierte las particularidades de los terminales reales a una definición normalizada de terminal de red y viceversa. El servidor TELNET interactúa con la aplicación, actuando como un sustituto del gestor del terminal, para que de esta forma el terminal remoto le parezca local a la aplicación. El tráfico entre el terminal del usuario y el servidor TELNET se lleva a cabo sobre una conexión TCP.

INTERFACES DE PROTOCOLO

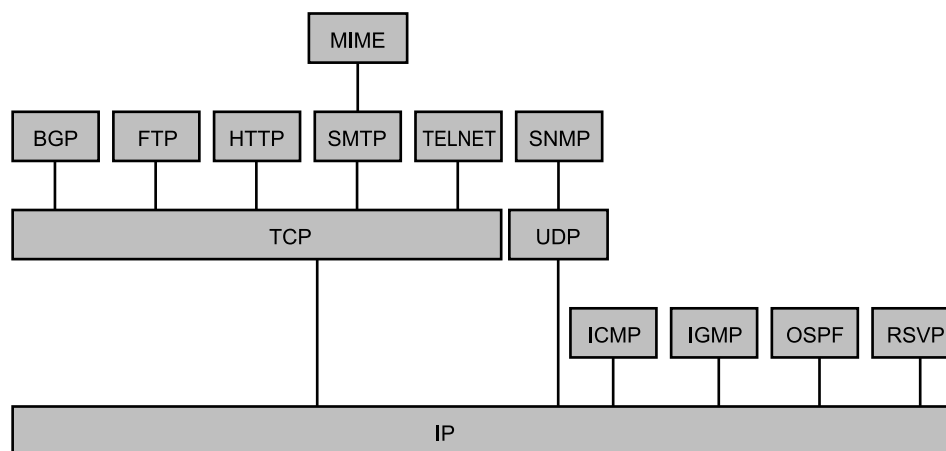
En la familia de protocolos TCP/IP cada capa interacciona con sus capas inmediatamente adyacentes. En el origen, la capa de aplicación utilizará los servicios de la capa extremo-a-extremo, pasándole los datos. Este procedimiento se repite en la interfaz entre la capa extremo-a-extremo y la capa internet, e igualmente en la interfaz entre la capa internet y la capa de acceso a la red. En el destino, cada capa entrega los datos a la capa superior adyacente.

La arquitectura de TCP/IP no exige que se haga uso de todas las capas. Como así se sugiere en la Figura 2.15, es posible desarrollar aplicaciones que invoquen directamente los servicios de cualquier capa. La mayoría de las aplicaciones requieren un protocolo extremo-a-extremo fiable y, por tanto, utilizan TCP. Otras aplicaciones de propósito específico no necesitan de los servicios del TCP. Algunas de estas, por ejemplo, el protocolo simple de gestión de red (SNMP), utilizan un protocolo extremo-a-extremo alternativo denominado protocolo de datagrama de usuario (UDP); otras, en cambio, incluso pueden usar el protocolo IP directamente. Las aplicaciones que no necesitan interconexión de redes y que no necesiten TCP pueden invocar directamente los servicios de la capa de acceso a la red.

2.5. LECTURAS RECOMENDADAS Y SITIOS WEB

Para el lector que esté interesado en profundizar en el estudio de TCP/IP, hay dos trabajos de tres volúmenes cada uno muy adecuados. Los trabajos de Comer y Stevens se han convertido en un clásico y son considerados como definitivos [COME00, COME99, COME01]. Los trabajos de Stevens y Wright son igualmente dignos de mención y más explícitos en la descripción del funcionamiento de los protocolos [STEV94, STEV96, WRIG95]. Un libro más compacto y útil como manual de referencia es [RODR02], en el que se cubre todo el espectro de protocolos relacionados con TCP/IP de una forma concisa y elegante, incluyendo la consideración de algunos protocolos no estudiados en los otros dos trabajos.

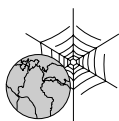
COME99 Comer, D., y Stevens, D. *Internetworking with TCP/IP, Volume II: Design Implementation, and Internals*. Upper Saddle River, NJ: Prentice Hall, 1994.



BGP (<i>Border gateway protocol</i>)	= Protocolo de pasarela fronteriza
FTP (<i>File transfer protocol</i>)	= Protocolo de transferencia de archivos
HTTP (<i>Hypertext transfer protocol</i>)	= Protocolo de transferencia de hipertexto
ICMP (<i>Internet control message protocol</i>)	= Protocolo de mensajes de control de Internet
IGMP (<i>Internet group management protocol</i>)	= Protocolo de gestión de grupos en Internet
IP (<i>Internet protocol</i>)	= Protocolo Internet
MIME (<i>Multipurpose internet mail extension</i>)	= Extensiones multipropósito de correo electrónico
OSPF (<i>Open shortest path first</i>)	= Protocolo del primer camino más corto disponible
RSVP (<i>Resource reservation protocol</i>)	= Protocolo de reserva de recursos
SMTP (<i>Simple mail transfer protocol</i>)	= Protocolo simple de transferencia de correo electrónico
SNMP (<i>Simple network management protocol</i>)	= Protocolo simple de gestión de red
TCP (<i>Transmission control protocol</i>)	= Protocolo de control de transmisión
UDP (<i>User datagram protocol</i>)	= Protocolo de datagrama de usuario

Figura 2.15. Algunos protocolos en la familia de protocolos TCP/IP.

- COME00 Comer D. *Internetworking with TCP/IP, Volume I: Principles, Protocols, and Architecture*. Upper Saddle River, NJ: Prentice Hall, 2000.
- COME01 Comer, D., y Stevens, D. *Internetworking with TCP/IP, Volume III: Client-Server Programming and Applications*. Upper Saddle River, NJ: Prentice Hall, 2001.
- RODR02 Rodríguez, A., et al., *TCP/IP: Tutorial and Technical Overview*. Upper Saddle River: NJ: Prentice Hall, 2002.
- STEV94 Stevens, W. *TCP/IP Illustrated, Volume 1: The Protocols*. Reading, MA: Addison-Wesley, 1994.
- STEV96 Stevens, W. *TCP/IP Illustrated, Volume 3: TCP for Transactions, HTTP, NNTP, and the UNIX(R) Domain Protocol*. Reading, MA: Addison-Wesley, 1996.
- WRIG95 Wright, G., y Stevens, W. *TCP/IP Illustrated, Volume 2: The Implementation*. Reading, MA: Addison-Wesley, 1995.



SITIO WEB RECOMENDADO

- «**Networking Links**»: ofrece una excelente colección de enlaces relacionadas con TCP/IP.

APÉNDICE 2A. EL PROTOCOLO TFTP (*TRIVIAL FILE TRANSFER PROTOCOL*)

En este apéndice se proporciona un resumen de la norma protocolo trivial para la transferencia de archivos (TFTP, *Trivial File Transfer Protocol*) de Internet, definido en el RFC 1350. Nuestro propósito aquí es ilustrar al lector sobre el uso de los elementos de un protocolo.

INTRODUCCIÓN A TFTP

TFTP es mucho más sencillo que la norma de Internet FTP (RFC 959). No hay mecanismos para controlar el acceso o la identificación de los usuarios. Por tanto, TFTP está sólo indicado para acceder a directorios públicos. Debido a su simplicidad, TFTP se implementa fácilmente y de una forma compacta. Por ejemplo, algunos dispositivos sin discos utilizan TFTP para descargar el código ejecutable para arrancar.

TFTP se ejecuta por encima de UDP. La entidad TFTP que inicia la transferencia lo hace enviando una solicitud de lectura o escritura en un segmento UDP al puerto 69 del destino. Este puerto se reconoce por parte del módulo UDP como el identificador del módulo TFTP. Mientras dura la transferencia, cada extremo utiliza un identificador para la transferencia (TID, *Transfer identifier*) como su número de puerto.

PAQUETES TFTP

Las entidades TFTP intercambian órdenes, respuestas y datos del archivo mediante paquetes, cada uno de los cuales se transporta en el cuerpo de un segmento UDP. TFTP considera cinco tipos de paquetes (véase Figura 2.17); los dos primeros bytes contienen un código que identifica el tipo de paquete de que se trata:

- **RRQ (*Read ReQuest packet*):** paquete para solicitar permiso para leer un archivo desde el otro sistema. Este paquete indica el nombre del archivo en una secuencia de bytes en ASCII³ terminada por un byte cero. Con este byte cero se indica a la entidad TFTP receptora que el nombre del archivo ha concluido. Este paquete también incluye un campo denominado modo, el cual indica cómo ha de interpretarse el archivo de datos: como una cadena de bytes ASCII o como datos de 8 bits en binario.

³ ASCII es la norma *American Standard Code for Information Interchange* especificada por el *American Standards Institute*. Asigna un patrón único de 7 bits a cada letra, usando el bit octavo como paridad. ASCII es equivalente al alfabeto de referencia internacional (IRA, *Internacional Referente Alphabet*), definido por la UIT-T en la recomendación T.50.

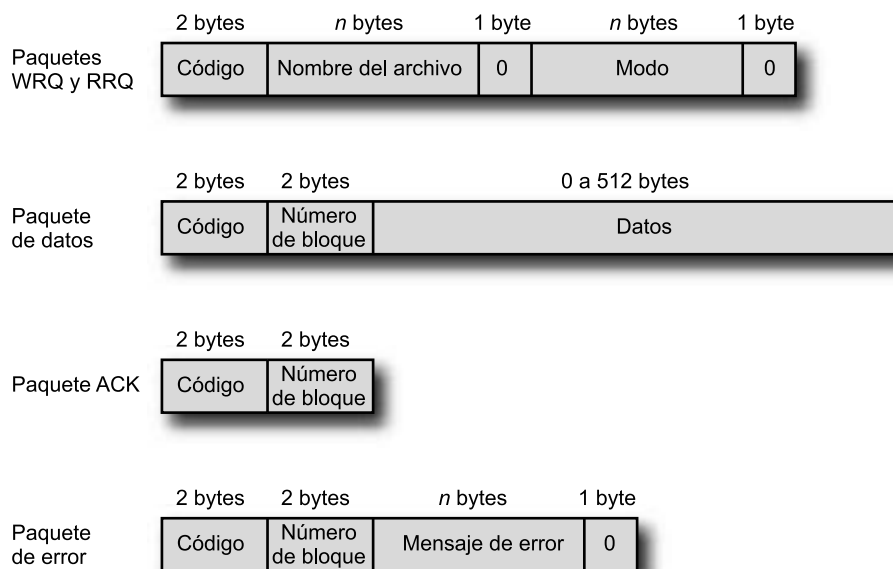


Figura 2.17. Formatos de los paquetes TFTP.

- **WRQ (Write ReQuest packet):** paquete para solicitar permiso para enviar un archivo al otro sistema.
- **Datos:** los bloques de datos se empiezan a enumerar desde uno y se incrementa su numeración por cada nuevo bloque de datos. Esta convención posibilita que el programa use un número para discriminar entre paquetes nuevos y duplicados. El campo de datos tiene una longitud entre cero y 512 bytes. Cuando es de 512 bytes, se interpreta como que no es el último bloque de datos. Cuando su longitud está comprendida entre cero y 511, indica el final de la transferencia.
- **ACK (Acknowledgement):** este paquete confirma la recepción de un paquete WRQ o de datos. El ACK de un paquete de datos contiene el número de bloque del paquete de datos confirmado. Un ACK de un WRQ contendrá un número de bloque igual a cero.
- **Error:** estos paquetes pueden corresponder a la confirmación de cualquier tipo de paquete. El código de error es un entero que indica la naturaleza del error (Tabla 2.4). El mensaje de error está destinado a ser interpretado por un humano, debiendo ser código ASCII. Al igual que todas las otras cadenas, se delimita finalmente con un byte cero.

Todos los paquetes, excepto los ACK duplicados (posteriormente explicados) y los usados para finalizar, tienen que ser confirmados. Para cualquier paquete se puede devolver un paquete de error. Si no hay errores, se aplica el siguiente convenio: los paquetes de datos o de tipo WRQ se confirman con un paquete ACK. Cuando se envía un RRQ, el otro extremo debe responder (siempre que no haya error) transmitiendo el archivo; por tanto, el primer bloque de datos sirve como confirmación del paquete RRQ. Hasta que no se concluya la transferencia del archivo, cada paquete ACK generado será seguido por un paquete de datos en el otro sentido. De esta forma los paquetes de datos sirven igualmente como confirmaciones. Un paquete de error podrá ser confirmado por cualquier tipo de paquete, dependiendo de las circunstancias.

Tabla 2.4. Códigos de error en TFTP.

Valor	Significado
0	No definido, ver mensaje de error (si lo hubiera)
1	Archivo no encontrado
2	Fallo en el acceso
3	Disco lleno o cuota excedida
4	Operación TFTP no válida
5	TID desconocido
6	El archivo ya existe
7	Usuario no existente

EJEMPLO DE TRANSFERENCIA

El ejemplo que se muestra en la Figura 2.18 corresponde a una transferencia sencilla desde A a B. Se supone que no ocurren errores. Es más, en la figura no se muestran los detalles acerca de la especificación de las opciones.

La operación comienza cuando el módulo TFTP del sistema A envía una solicitud de escritura (WRQ) al módulo TFTP del sistema B. El paquete WRQ se transporta en el cuerpo de un segmento UDP. La solicitud de escritura incluye el nombre del archivo (en el ejemplo XXX) y un octeto que indica el modo, binario u octetos. En la cabecera UDP, el puerto destino es el 69, el cual avisa a la entidad UDP receptora que el mensaje está dirigido a la aplicación de TFTP. El número del puerto origen es un TID seleccionado por A, en el ejemplo 1511. El sistema B está preparado para aceptar el archivo. Por tanto, devuelve un ACK con número de bloque 0. En la cabecera UDP, el puerto destino es 1511, el cual habilita a la entidad UDP en A a encaminar el paquete recibido al módulo TFTP, el cual podrá cotejar este TID con el TID del WRQ. El puerto origen es un TID seleccionado por B para la transferencia del archivo, 1660 en el ejemplo.

Siguiendo con el ejemplo, se procede a la transferencia del archivo. El envío consiste en uno o más paquetes desde A, cada uno de los cuales ha de ser confirmado por B. El último paquete de datos contiene menos de 512 bytes de datos, lo que indica el fin de la transferencia.

ERRORES Y RETARDOS

Si TFTP funciona sobre una red o sobre una internet (por oposición a un enlace directo de datos), es posible que ciertos paquetes se pierdan. Debido a que TFTP funciona sobre UDP, el cual no proporciona un servicio con entrega garantizada, se necesita en TFTP un mecanismo que se encargue de tratar con estos posibles paquetes perdidos. En TFTP se usa una técnica habitual basada en expiración de temporizadores. Supóngase que A envía un paquete a B que requiere ser confirmado. (es decir, cualquier paquete que no sea un ACK duplicado o uno utilizado para terminar). Cuando A envía el paquete, inicia un temporizador. Si el temporizador expira antes de que se reciba de B

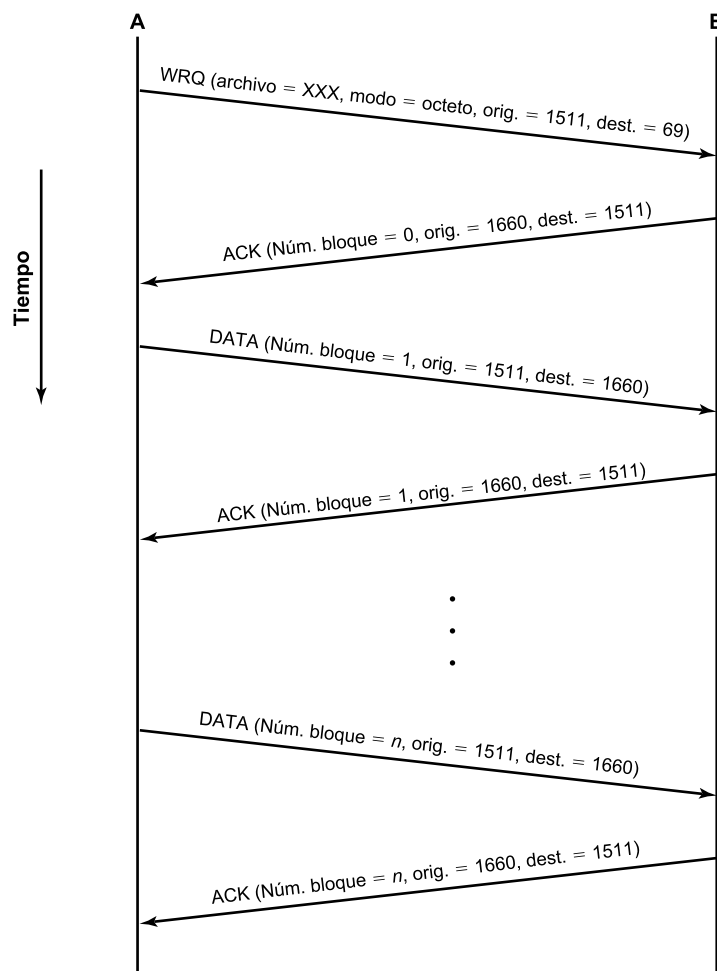


Figura 2.18. Ejemplo de funcionamiento de TFTP.

la confirmación, A retransmitirá el mismo paquete anterior. Si de hecho el paquete original se perdió, la retransmisión será la primera instancia del paquete que reciba B. Si el paquete original no se perdió, pero sí la confirmación de B, B recibirá dos copias del mismo paquete, confirmando ambas. Debido al uso de número de bloques, esta última contingencia no introducirá confusión alguna. La única excepción sería el caso de ACK duplicados. En este caso, el segundo ACK se ignorará.

SINTAXIS, SEMÁNTICA Y TEMPORIZACIÓN

En el Apartado 2.1 se mencionó que las características o aspectos clave de cualquier protocolo se pueden clasificar en sintaxis, semántica y temporización. Estas categorías se pueden identificar fácilmente en TFTP. El formato de los distintos paquetes TFTP constituye la **sintaxis** del protocolo. La **semántica** del protocolo consiste en las definiciones de cada uno de los tipos de paquetes y códigos de error. Finalmente, el orden en que se intercambian los paquetes, la numeración de los bloques y el uso de temporizadores son todos ellos aspectos relativos a la **temporización** de TFTP.